

125. Valid palindrome:

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` if it is a **palindrome**, or `false` otherwise.

Example 1:

Input: `s = "A man, a plan, a canal: Panama"`

Output: `true`

Explanation: "amanaplanacanalpanama" is a palindrome.

Example 2:

Input: `s = "race a car"`

Output: `false`

Explanation: "raceacar" is not a palindrome.

Example 3:

Input: `s = ""`

Output: `true`

Explanation: `s` is an empty string "" after removing non-alphanumeric characters.

Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- `1 <= s.length <= 2 * 105`
- `s` consists only of printable ASCII characters.

Solution: time = O(n) and space = O(n)

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
```

```
s = ''.join(c.lower()
for c in s if c.isalnum())
return s == s[::-1]
```

Optimal - Two Pointer - time = $O(n)$ and space = $O(1)$

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        left, right = 0, len(s) - 1

        while left < right:
            while left < right and not s[left].isalnum():
                left += 1
            while left < right and not s[right].isalnum():
                right -= 1

            if s[left].lower() != s[right].lower():
                return False

            left += 1
            right -= 1

        return True
```